	Workshop Implementation Phase **Inter-entity Collision**	
	Subject: Project 1A Class(es): 1A Team: Project 1A	Teaching Unit: Algorithms & Programming Academic Year: 2025–2026

Objectives:

Focus on the implementation phase by referring to the CDIO initiative [Implement]:

✔ Transform the design into a product
✔ Coding
✔ Testing
✔ Validation

1. Collision Theory

A **collision** is a direct impact between two objects.

Collision detection is a mechanism used to identify prohibited movements or those requiring special handling during the simulation of moving objects.

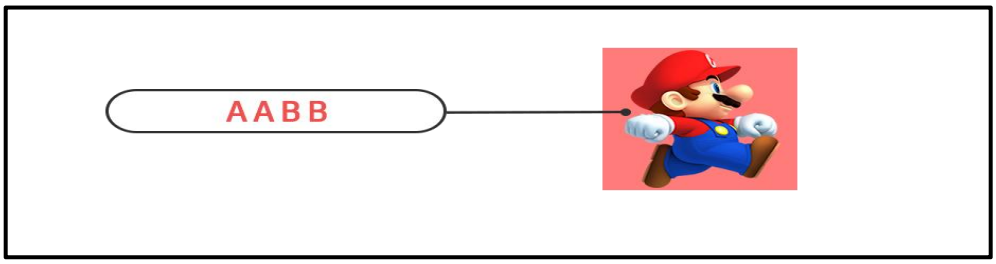
Collisions are a fundamental aspect of any action game or animation.

Example: Check if a character touches an enemy? Touches the ground? etc.

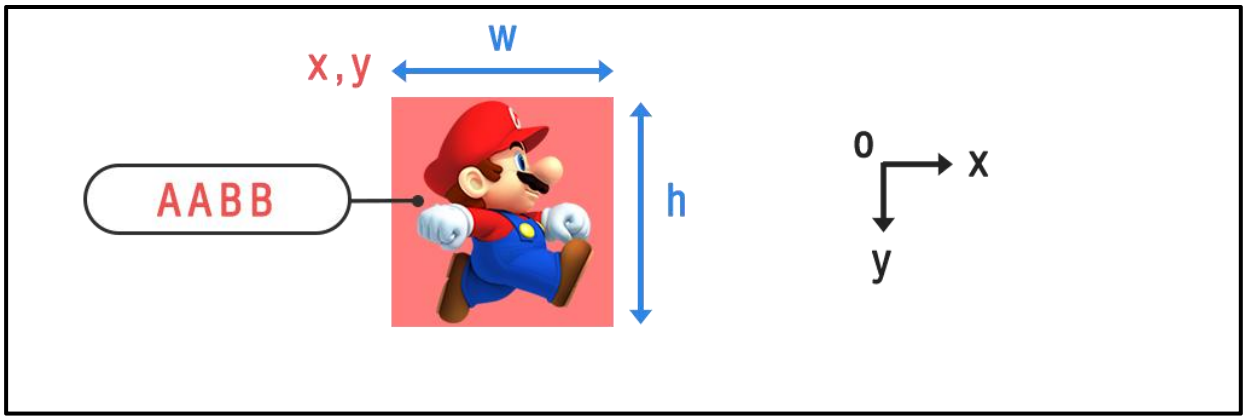
2. Collision Between Two Objects

AABB (Axis Aligned Bounding Box) :

This refers to a rectangle aligned with the axes, meaning its sides are parallel to the x and y axes of your coordinate system (screen).



An AABB can be defined by 4 parameters: the x and y positions of its top-left corner (in 2D, the Y axis points downward), as well as its width **w** and height **h**.

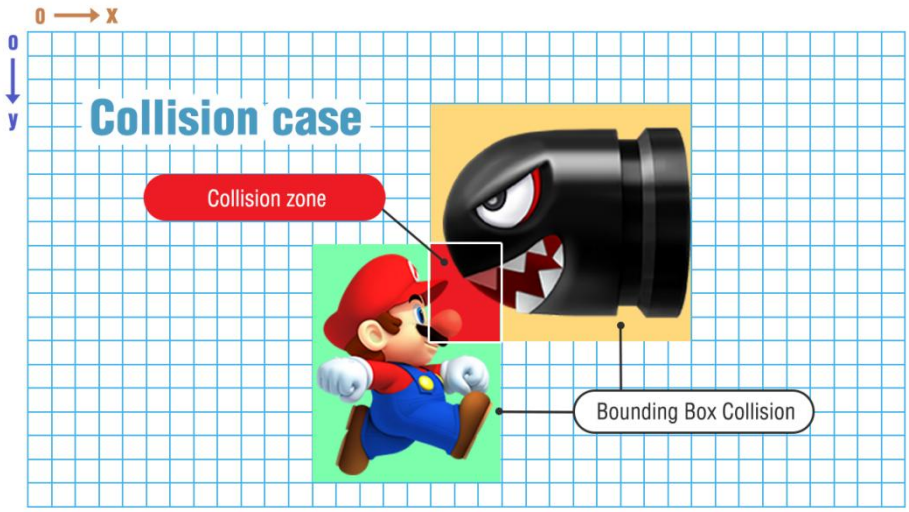


This leads to the declaration of an AABB variable of type SDL_Rect, characterized by the following fields:

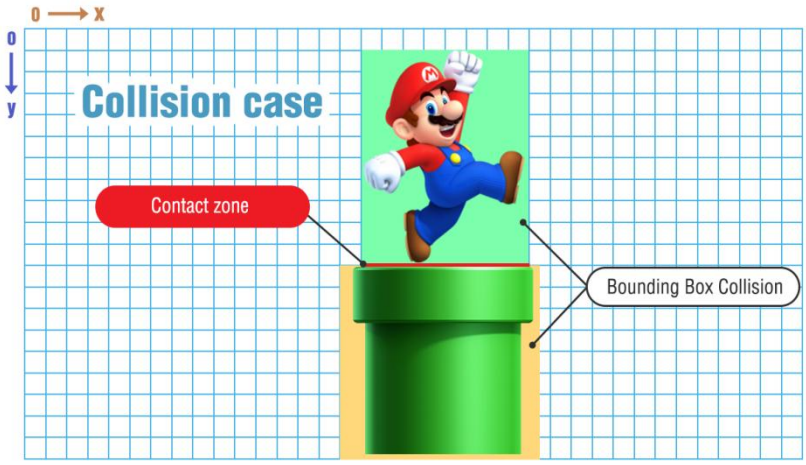
```
int x ;
int y ;
int w ;
int h ;
```

A collision is detected :

- ✔ If two entities come into contact



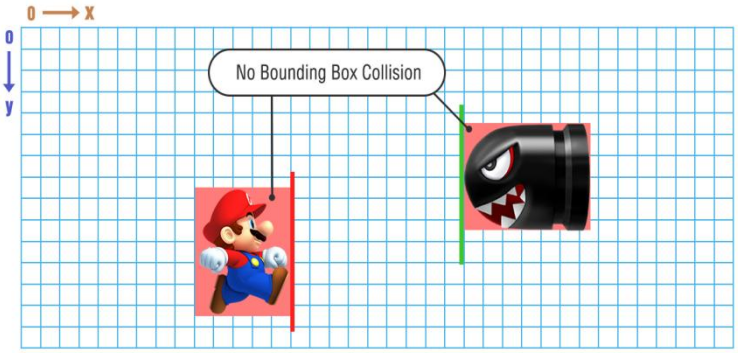
✔ If two entities coincide from a contact area.



In this workshop, we will focus on the second case:

The principle of collision between two AABB-type objects is illustrated by the following figures:

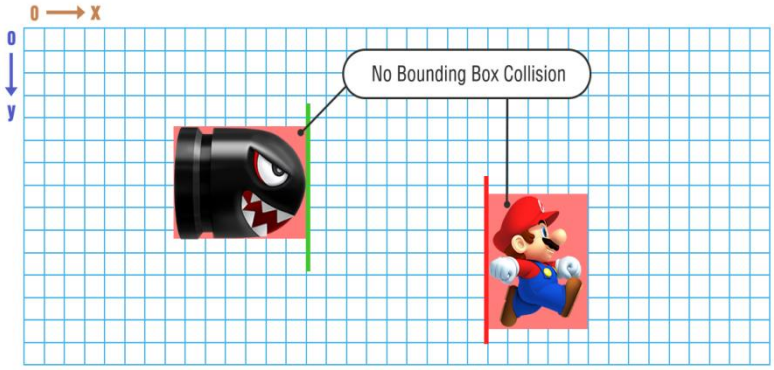
Case 1 :



✓ The character is completely to the left of the enemy:

```
character.x+ character. w < enemy. x
```

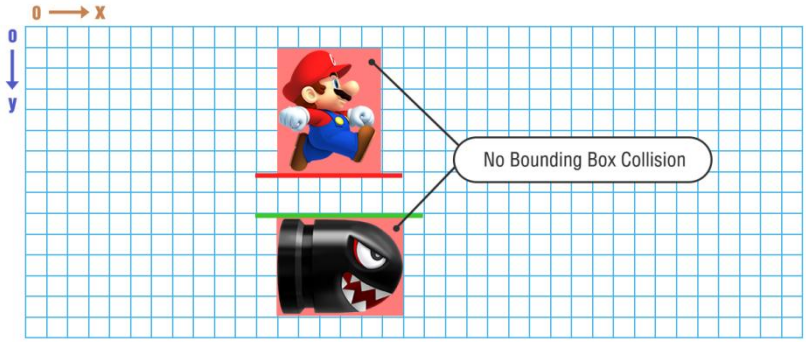
Case 2 :



✓ The character is completely to the right of the enemy :

```
character.x  > enemy. x +enemy. w
```

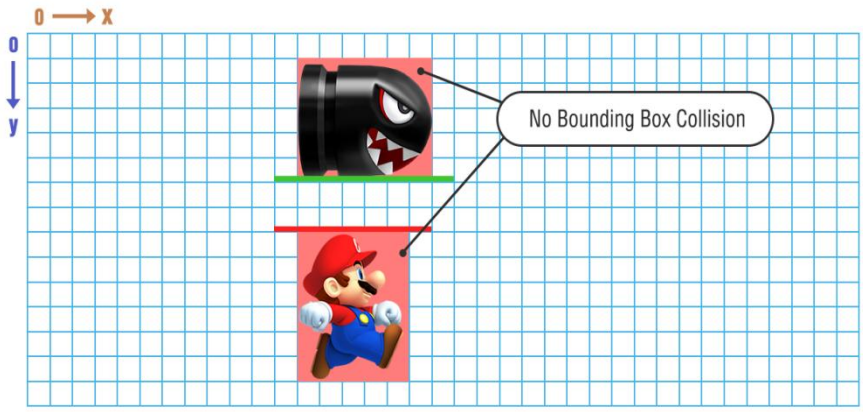
Case 3 :



✓ The character is completely above the enemy :

```
character.y + character.h < enemy. y
```

Case 4 :



✓ The character is completely below the enemy :

```
character.y > enemy. y + enemy. h
```

The rule then becomes :

IF
(Character.x + Character.w < Enemy.x) **OR**
(Character.x > Enemy.x + Enemy.w) **OR**
(Character.y + Character.h < Enemy.y) **OR**
(Character.y > Enemy.y + Enemy.h)
THEN
  Collision = 0 // no collision
ELSE
  Collision = 1
END IF

3. Example of a scenario for integrating collision detection into the game loop:

When a collision is detected (e.g., collision with an entity), the score and the number of lives will be updated. (See the algorithm below)

BEGIN
// Initialize resources
.....
// Initialize score and number of lives (score, lives_count, ...)
While (continue)
    If (collision) **THEN**

```
        lives_count--
        score--
    End If
    If ((lives_count == 0) or (score < 0)) THEN
        continue = 0
    End If
End While
// Release resources
END
```

Work to do:

Apply the principle of inter-entity collision in the implementation of your game.